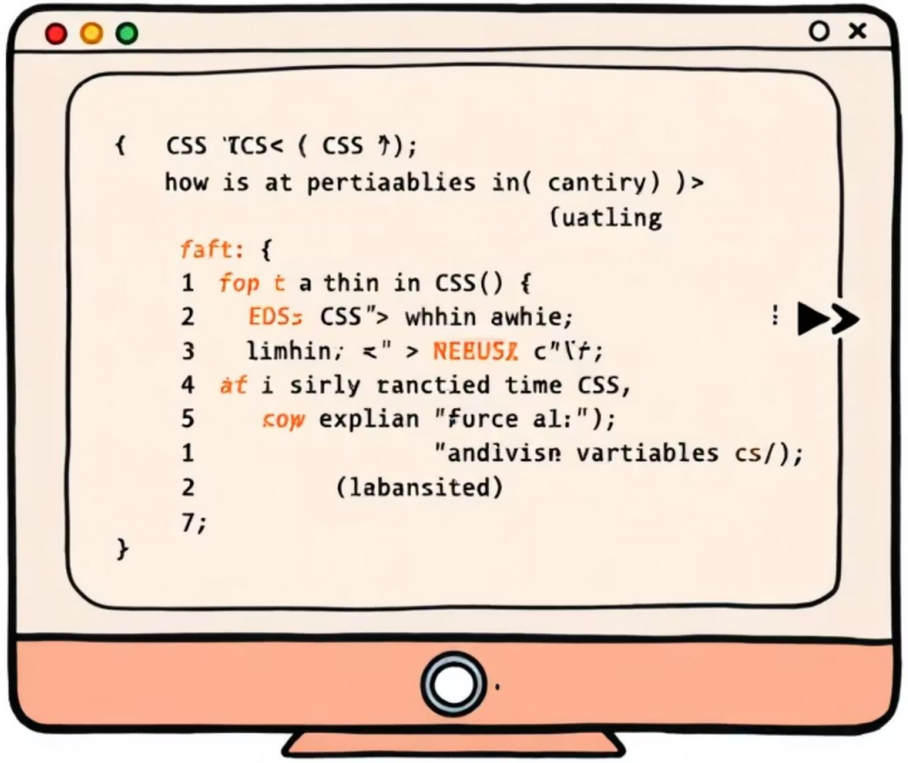


Variables en CSS (Custom Properties)

Las **variables CSS**, también conocidas como *custom properties*, son una característica fundamental del CSS moderno que permite almacenar valores reutilizables en toda la hoja de estilos. Son especialmente útiles para mantener la coherencia visual, facilitar el mantenimiento del código y crear sistemas de diseño escalables y profesionales.

CSS

¿Qué son las Variables CSS?

Las variables CSS permiten definir valores una sola vez y reutilizarlos en múltiples lugares del código. Se declaran utilizando el prefijo `--` y se acceden mediante la función `var()`. Esta funcionalidad transforma completamente la forma en que escribimos y mantenemos nuestros estilos.

Dónde se definen

Normalmente se declaran en el selector `:root`, que representa el elemento raíz del documento HTML. Esto las convierte en variables globales accesibles desde cualquier parte del CSS:

```
:root {
  --color-principal: #1D3557;
  --color-acento: #E63946;
  --espaciado-base: 1rem;
  --fuente-titulos: 'Helvetica', sans-serif;
}
```

Cómo se usan

Para acceder al valor de una variable, utilizamos la función `var()` seguida del nombre de la variable:

```
h1 {
  color: var(--color-principal);
  font-family: var(--fuente-titulos);
  margin-bottom: var(--espaciado-base);
}
```

Ventajas clave

- Cambias un valor y se actualiza toda la web
- Facilitan enormemente el mantenimiento
- Permiten crear temas (claro/oscuro)
- Código más limpio y profesional
- Reducen repetición de código

CSS

Variables CSS en Acción

Las variables CSS brillan especialmente cuando necesitas mantener coherencia visual en proyectos grandes o implementar funcionalidades avanzadas como temas personalizables. Veamos un ejemplo práctico que muestra su verdadero potencial:

Definición

Declaramos las variables en `:root` con valores iniciales que representan nuestro tema principal

```
:root {
  --fondo: #ffffff;
  --texto: #000000;
  --primario: #1D3557;
}
```

Aplicación

Usamos las variables en nuestros selectores mediante `var()`

```
body {
  background: var(--fondo);
  color: var(--texto);
}
```

Tema oscuro

Cambiamos los valores con JavaScript o CSS para crear un tema alternativo

```
[data-theme="dark"] {
  --fondo: #1a1a1a;
  --texto: #ffffff;
}
```

Consejo profesional: Organiza tus variables por categorías (colores, tipografía, espaciado) y documéntalas para que otros desarrolladores entiendan su propósito. Esto mejora significativamente la colaboración en equipos.

CSS

Position: Fixed

¿Qué hace position: fixed?

La propiedad `position: fixed` coloca un elemento en una posición **fija respecto a la ventana del navegador** (viewport). A diferencia de otros posicionamientos, el elemento permanece visible en la misma posición incluso cuando el usuario hace scroll por la página.

Esta característica lo convierte en la herramienta perfecta para elementos de interfaz que necesitan estar siempre accesibles, como menús de navegación, botones de acción flotantes o avisos importantes.

Sintaxis básica

```
.boton-fixed {
  position: fixed;
  bottom: 20px;
  right: 20px;
  z-index: 1000;
}
```

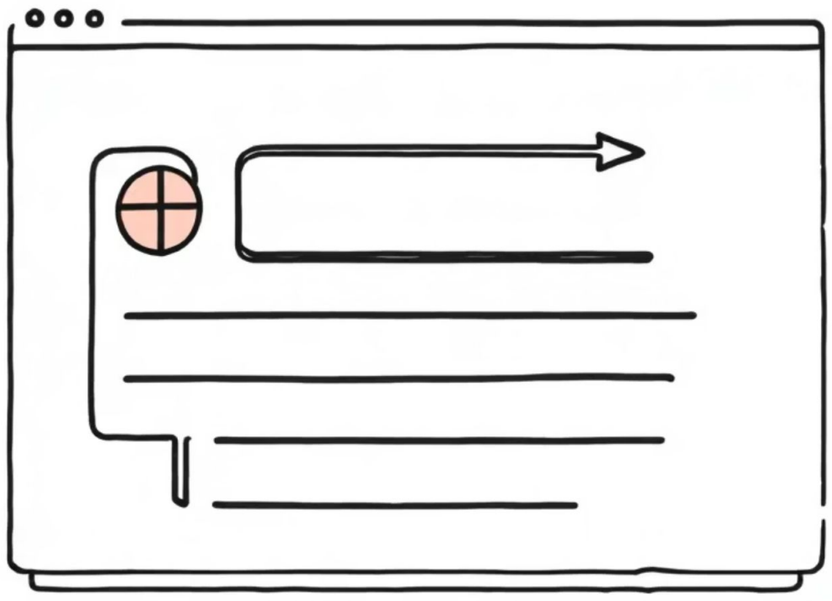
Usos habituales

Botones "volver arriba": facilitan la navegación en páginas largas

Menús de navegación: mantienen el acceso a las secciones principales

Chats flotantes: atención al cliente siempre disponible

Avisos legales: cookies y políticas de privacidad



CSS

Implicaciones de Position: Fixed

Aunque `position: fixed` es una herramienta poderosa, requiere un uso cuidadoso y considerado. Al sacar elementos del flujo normal del documento, puede generar problemas inesperados si no se maneja correctamente.

Sale del flujo normal

El elemento fijo no ocupa espacio en el documento, lo que significa que otros elementos se comportan como si no existiera. Esto puede causar superposiciones no deseadas con el contenido principal.

Puede tapar contenido

Sin un `z-index` adecuado y márgenes en el contenido principal, los elementos fijos pueden ocultar información importante. Es fundamental reservar espacio para ellos.

Problemas en móviles

Los dispositivos móviles presentan desafíos únicos: el teclado virtual puede tapar elementos fijos, la pantalla reducida hace que ocupen proporcionalmente más espacio, y pueden dificultar la interacción táctil.

Solución para dispositivos móviles

Para evitar problemas en pantallas pequeñas, es recomendable usar media queries que oculten o reposicionen elementos fijos en móviles:

```
@media (max-width: 600px) {
  .boton-fixed {
    /* Opción 1: Ocultarlo */
    display: none;

    /* Opción 2: Reposicionarlo */
    bottom: 10px;
    right: 10px;
    padding: 0.5rem;
  }
}
```

Mejor práctica: Considera usar `position: sticky` como alternativa en móviles. Se comporta como `relative` hasta que alcanza un umbral de scroll, entonces actúa como `fixed`.

CSS

Float: El Concepto Clásico

La propiedad `float` es una de las técnicas más antiguas de CSS, diseñada originalmente para crear layouts donde el texto fluye alrededor de imágenes, similar a las maquetaciones de revistas y periódicos impresos. Aunque su uso para crear estructuras de página completas ha quedado obsoleto, entender `float` es importante para trabajar con código legacy y para su propósito original.

Años 90-2000

Float se usaba para todo el layout de páginas web. Era la única herramienta disponible para crear columnas y estructuras complejas.

1

2

2010-2015

Aparecen frameworks como Bootstrap que sistematizan el uso de floats para grids, pero surgen los primeros problemas de mantenimiento.

3

2015-presente

Flexbox y Grid revolucionan el CSS. Float vuelve a su propósito original: imágenes con texto.

Uso correcto hoy en día

```
/* Imagen flotante con texto alrededor */
img {
  float: left;
  margin-right: 10px;
  margin-bottom: 10px;
  max-width: 300px;
}
```

Este es el uso para el que `float` fue diseñado: permitir que las imágenes se integren naturalmente dentro del flujo de texto, creando layouts editoriales elegantes.

Problemas de Float y Alternativas Modernas

Aunque `float` sirvió bien a la web durante décadas, presenta limitaciones significativas que llevaron al desarrollo de tecnologías más robustas. Es fundamental entender estos problemas para apreciar las soluciones modernas.

Sale del flujo normal

Los elementos flotados no contribuyen a la altura de su contenedor padre, causando el colapso del mismo. Esto genera problemas visuales inesperados y requiere soluciones adicionales.

Rompe alturas de contenedores

El famoso "problema del contenedor colapsado" obligaba a usar trucos como el clearfix, añadiendo complejidad innecesaria al código.

Requiere clearfix

Necesitas añadir código extra para "limpiar" los floats y restaurar el comportamiento normal del flujo de documento.

Difícil de mantener

Los layouts basados en float son frágiles y propensos a romperse con cambios pequeños, dificultando el mantenimiento a largo plazo.

El famoso "Clearfix"

```
.clearfix::after {
  content: "";
  display: block;
  clear: both;
}
```

¿Se usa float hoy en día?

NO para layouts completos: Flexbox y Grid son superiores en todos los aspectos

SÍ para imágenes con texto: Su propósito original sigue siendo válido

SÍ en código legacy: Necesitarás entenderlo para mantener proyectos antiguos

Usa en su lugar:

`display: flex` → para layouts unidimensionales (filas o columnas)

`display: grid` → para layouts bidimensionales (filas Y columnas)

`position` → para posicionamiento específico de elementos

CSS

Media Queries: Diseño Responsive

¿Qué son las Media Queries?

Las **media queries** son la piedra angular del diseño web responsive. Permiten aplicar estilos CSS diferentes según las características del dispositivo que esté visualizando la página, principalmente el ancho de la pantalla. Esto hace posible crear una única base de código que se adapta elegantemente a móviles, tablets, portátiles y monitores de escritorio.

Gracias a las media queries, podemos transformar completamente el layout, la tipografía, los espaciados y cualquier aspecto visual para optimizar la experiencia en cada contexto de uso.

Móviles (hasta 480px)

Layout de una columna, tipografía base, menús colapsados. Prioridad absoluta a la simplicidad y el contenido esencial.

Tablets (481px - 768px)

Posibilidad de dos columnas, espaciados intermedios. Balance entre densidad de información y usabilidad táctil.

Portátiles (769px - 992px)

Layouts más complejos, sidebars opcionales. Aprovechamos el espacio extra para mostrar más información simultáneamente.

Escritorio (993px+)

Diseño completo, múltiples columnas, espaciados amplios. Experiencia visual rica y optimizada para mouse y teclado.

CSS

Sintaxis y Estrategias de Media Queries

Sintaxis básica

```
@media (max-width: 600px) {
  body {
    background: lightblue;
    padding: 1rem;
  }

  h1 {
    font-size: 1.5rem;
  }
}
```

Breakpoints comunes

Tamaño	Uso típico
480px	Móviles pequeños (iPhone SE, etc.)
768px	Tablets y móviles grandes en horizontal
992px	Portátiles y pantallas medianas
1200px	Escritorio y pantallas grandes
1400px+	Monitores extra grandes y 4K

Dos filosofías: Mobile First vs Desktop First

Mobile First (Recomendado)

Se diseña primero para móvil y luego se mejora progresivamente para pantallas más grandes usando `min-width`:

```
/* Estilos base para móvil */
.contenedor {
  width: 100%;
}

/* Mejorar para escritorio */
@media (min-width: 768px) {
  .contenedor {
    width: 1200px;
  }
}
```

Desktop First

Se diseña primero para escritorio y luego se adapta a pantallas más pequeñas usando `max-width`:

```
/* Estilos base para escritorio */
.contenedor {
  width: 1200px;
}

/* Adaptar a móvil */
@media (max-width: 768px) {
  .contenedor {
    width: 100%;
  }
}
```

Nota importante: Estos breakpoints son orientativos. Lo ideal es definirlos según las necesidades específicas de tu diseño, no según dispositivos concretos.

Mobile First es la aproximación recomendada por varias razones: la mayoría del tráfico web viene de móviles, fuerza a priorizar el contenido esencial, y mejora el rendimiento al cargar menos CSS en dispositivos con menos recursos.

CSS

Buenas Prácticas con Media Queries

1

Organiza las media queries al final

Mantén todas las media queries agrupadas al final de tu archivo CSS, o mejor aún, en archivos separados. Esto facilita enormemente el mantenimiento y permite encontrar rápidamente los ajustes responsive.

```
/* Todos los estilos base primero */
.header { ... }
.main { ... }

/* Media queries al final */
@media (max-width: 768px) {
  .header { ... }
  .main { ... }
}
```

2

No abuses de ellas

Diseña layouts fluidos que se adapten naturalmente a diferentes tamaños usando unidades relativas (%, rem, em) y técnicas como Flexbox y Grid. Las media queries deben ser puntos de inflexión, no ajustes constantes.

3

Diseña pensando en móvil

Adopta la metodología Mobile First. Diseña primero la experiencia móvil, asegurándote de que el contenido esencial está presente y accesible. Luego, mejora progresivamente para pantallas más grandes añadiendo características y complejidad.

4

Cambia layout, no solo colores

Las media queries no son solo para ajustar tamaños de fuente o colores. Úsalas para transformar completamente el layout: cambiar de columna única a múltiples columnas, mostrar/ocultar elementos, reorganizar el orden visual del contenido.

Ejemplo práctico completo

```
/* Mobile First base */
.grid {
  display: grid;
  gap: 1rem;
  grid-template-columns: 1fr;
}

/* Tablet */
@media (min-width: 768px) {
  .grid {
    grid-template-columns: 1fr 1fr;
    gap: 1.5rem;
  }
}

/* Desktop */
@media (min-width: 1200px) {
  .grid {
    grid-template-columns: repeat(3, 1fr);
    gap: 2rem;
  }
}
```

Testing responsive

- Usa las DevTools del navegador para simular diferentes dispositivos
- Prueba en dispositivos reales siempre que sea posible
- No te fíes solo de breakpoints estándar
- Comprueba tanto en orientación vertical como horizontal
- Verifica el rendimiento en dispositivos móviles reales

CSS